

MarketMind AI

Technical Architecture Overview — v3

LangChain Dependency Injection Refactor | LangSmith Full Observability
Saudi Arabia GTM Intelligence Platform | Classification: Internal Technical Reference

Revision Summary — v2 → v3

This document supersedes the v2 Technical Overview. The core pipeline logic, Pydantic schemas, safety guardrails, and ChromaDB persistence are unchanged. The following architectural components were refactored:

Component	Change
app.py — <code>_get_clients()</code>	Now instantiates one shared ChatOpenAI instance and returns it as a third value alongside <code>openai_client</code> and <code>tavily_client</code> .
app.py — all 4 agent call sites	Unpack <code>llm</code> from <code>resolve_clients()</code> and pass as <code>llm=llm</code> keyword argument to every agent constructor.
ResearchAgent. <code>__init__</code>	Accepts optional <code>llm: ChatOpenAI</code> . Uses <code>self.llm.invoke()</code> for <code>synthesise_report</code> and <code>build_search_queries</code> . <code>openai_client</code> retained for embeddings only.
StrategyAgent. <code>__init__</code>	Accepts optional <code>llm: ChatOpenAI</code> . Uses <code>self.llm.with_structured_output(GTMStrategyDocument).invoke()</code> for strategy generation.
ContentAgent. <code>__init__</code>	Accepts optional <code>llm: ChatOpenAI</code> . Uses <code>self.llm.with_structured_output(ContentPackage)</code> for draft and refinement. Critique uses <code>self.llm.invoke()</code> .
BrandAlignmentAgent. <code>__init__</code>	Accepts optional <code>llm: ChatOpenAI</code> . Uses <code>self.llm.with_structured_output(RefinedContent)</code> for the compliance audit call.
LangSmith Tracing	All ChatOpenAI calls now emit automatic traces to LangSmith. Token counts, cost, and latency appear per-call without any manual instrumentation.
Embeddings	Unchanged. All four agents continue to use <code>self.openai_client.embeddings.create()</code> directly. ChatOpenAI does not support embeddings.

Component	Change
Pydantic Schemas	Completely unchanged. CompanyProfile, GTMStrategyDocument, ContentPackage, RefinedContent, and all sub-schemas are identical to v2.
Safety Guardrails	Completely unchanged. All three layers (input validation, generation guardrails, Brand Alignment audit) are identical to v2.

1. High-Level Metrics — At a Glance (v3)

The following table reflects the system state after the LangChain Dependency Injection refactor. Rows marked with a change indicator differ from the v2 baseline.

Component	Count / Status (v3)
AI Models	2 — GPT-4o-mini via ChatOpenAI (LangChain) for inference; text-embedding-3-small via OpenAI SDK for embeddings
LLM Invocation Pattern [CHANGED]	self.llm.invoke() and self.llm.with_structured_output().invoke() — all 4 agents; raw OpenAI SDK retained for embeddings only
Agent Classes	4 — ResearchAgent, StrategyAgent, ContentAgent, BrandAlignmentAgent (all accept injected llm parameter)
Integrated External APIs	3 — OpenAI API (via ChatOpenAI + raw SDK), Tavily Search API, LangSmith Tracing API
Dependency Injection [NEW]	One shared ChatOpenAI instance created in app.py._get_clients(), cached via @st.cache_resource, passed to all 4 agents
RAG (Retrieval-Augmented Generation)	Active — unchanged from v2. ChromaDB ID-based retrieval between pipeline stages.
Embeddings	Active — unchanged. text-embedding-3-small via OpenAI SDK. Not routed through ChatOpenAI.
Streamlit Frontend	Active — unchanged. Session-state substage machine, st.tabs, st.container(border=True).
Guardrails & Safety Layers	Active — unchanged. 3-layer defence: input injection guard, generation prohibitions, Brand Alignment audit.

Component	Count / Status (v3)
LangSmith Tracing [UPGRADED]	Fully active. All ChatOpenAI.invoke() calls emit automatic traces. Token count, cost (USD), and latency per span appear in LangSmith dashboard.
Token & Cost Tracking (sidebar) [NEW]	Active. _track_tokens() accumulates totals in session_state. Live sidebar panel shows prompt tokens, completion tokens, total tokens, estimated cost.
Persistent Memory	Active — unchanged. ChromaDB PersistentClient. 4 collections. chroma_db/ secured via .gitignore.

2. Technical Deep-Dive — What Changed and Why

2.1 The Core Problem: Why Direct OpenAI Client Calls Did Not Appear in LangSmith

Prior to this refactor, all four agents used the raw OpenAI Python SDK directly: `self.openai_client.chat.completions.create()` and `self.openai_client.beta.chat.completions.parse()`. While the `LANGCHAIN_TRACING_V2=true` environment variable was correctly set in `app.py`, LangChain's tracing hooks only intercept calls made through its own abstractions — specifically through `ChatOpenAI`, `ChatPromptTemplate.pipe()`, or `RunnableSequence`. A raw OpenAI SDK call is invisible to LangSmith regardless of how many environment variables are set.

The result was an empty LangSmith dashboard: the project existed, the API key was valid, but no runs appeared because all LLM traffic was bypassing LangChain's instrumented code paths entirely. This is the architectural gap that the Dependency Injection refactor closes.

2.2 The Solution: ChatOpenAI Dependency Injection

What the new initialisation flow looks like

A single `ChatOpenAI` instance is now created once in `app.py` inside the `@st.cache_resource-`decorated `_get_clients()` function. This function is called once per Streamlit session (cached on the `openai_key` + `tavily_key` pair) and returns a triple:

- `openai_client`: `OpenAI` — the raw OpenAI SDK client, retained exclusively for embedding calls.
- `tavily_client`: `TavilyClient` — the Tavily search client, unchanged.
- `llm`: `ChatOpenAI` — the new shared LangChain inference client, injected into all four agents.

The ChatOpenAI instance is configured with `model='gpt-4o-mini'` and `temperature=0.4` as the session-level default. Each agent may override temperature at the invocation level when its task requires different settings (e.g., the Research Agent uses `temperature=0.35` for factual synthesis; the Content Agent uses `temperature=0.5` for creative generation).

How each agent receives the injected instance

All four agent constructors were updated to accept an optional `llm: ChatOpenAI | None = None` parameter. This design choice — optional rather than required — preserves backwards compatibility and allows agents to be instantiated independently in unit tests without requiring a live LangChain installation. The constructor logic follows a strict priority order:

- Priority 1 — Use the injected `llm` if `app.py` passed one (production path).
- Priority 2 — Create a local ChatOpenAI instance if LangChain is installed but no `llm` was injected (standalone testing path).
- Priority 3 — Set `self.llm = None` if LangChain is not installed; all LLM calls fall back to `self.openai_client` (graceful degradation path).

This three-tier fallback ensures the pipeline never crashes due to a missing dependency, while still guaranteeing that the production path always uses the shared, instrumented ChatOpenAI instance.

2.3 From `openai_client.create()` to `self.llm.invoke()`

Text generation calls — `self.llm.invoke()`

For calls that return plain text (synthesise_report in ResearchAgent, the self-reflection critique in ContentAgent), the pattern is:

- Build a list of LangChain message objects: `SystemMessage(content=...)` and `HumanMessage(content=...)`.
- Call `self.llm.invoke(messages)` which returns an `AIMessage` object.
- Extract the text via `response.content.strip()`.

The `invoke()` call is the instrumentation point. LangChain intercepts it, measures latency, counts tokens via the OpenAI usage response field, computes cost using the model's known pricing, and emits a trace span to LangSmith — all automatically, with no additional code required.

Structured output calls — `self.llm.with_structured_output()`

For calls that must return a typed Pydantic object (GTMStrategyDocument, ContentPackage, RefinedContent), the pattern is:

- Bind the schema: `structured_llm = self.llm.with_structured_output(PydanticSchema)`.
- Invoke: `result = structured_llm.invoke(messages)`.
- The result is a fully validated Pydantic instance — no manual JSON parsing required.

`with_structured_output()` uses OpenAI's function-calling / JSON schema mode under the hood, which is the same mechanism the previous `beta.chat.completions.parse()` call used. The behaviour is identical from an output quality perspective. The key difference is that the call now flows through LangChain's instrumented layer, making it visible to LangSmith.

What stays on the raw OpenAI SDK

Embeddings are not routed through ChatOpenAI. The text-embedding-3-small model is called via `self.openai_client.embeddings.create()` in all four agents, exactly as in v2. LangChain's ChatOpenAI wrapper does not expose an embeddings interface — that would require `langchain_openai.OpenAIEmbeddings`, which would introduce a separate dependency. Since embedding calls are not LLM inference calls and do not need token-cost attribution in the same granular way, retaining them on the raw SDK is the correct architectural decision.

2.4 LangSmith Observability — What You Now See

Automatic trace emission

Because all four agents now call the LLM through the shared ChatOpenAI instance, every inference call produces a LangSmith trace span automatically. No `@traceable` decorators, no manual logging, no callback handlers are required. LangChain instruments `ChatOpenAI.invoke()` at the library level — any call through it is traced when `LANGCHAIN_TRACING_V2=true` and `LANGCHAIN_API_KEY` are set in the environment.

What each trace span contains

- Span name — derived from the agent name injected via `@traceable` wrappers in `app.py` (`Stage1_SynthesiseReport`, `Stage2_StrategyAgent`, etc.).
- Input — the full rendered prompt (system + human messages) sent to the model.
- Output — the full model response, including structured output where applicable.
- Token counts — `prompt_tokens`, `completion_tokens`, `total_tokens` extracted from the OpenAI usage field.
- Latency — wall-clock time from invocation to response in milliseconds.
- Estimated cost — computed from token counts using GPT-4o-mini pricing (\$0.15/1M prompt, \$0.60/1M completion).
- Model — `gpt-4o-mini`, visible per span for future multi-model comparisons.

The sidebar cost panel

In parallel with LangSmith, `app.py` accumulates token counts locally using the `_track_tokens()` helper and the `_TrackingClient` proxy. This provides an immediate cost estimate in the Streamlit sidebar without requiring the user to navigate to the LangSmith dashboard mid-session. The sidebar panel shows: prompt tokens, completion tokens, total

tokens, estimated cost in USD, and a collapsible per-call breakdown listing each agent call with its individual token split and cost.

2.5 What Did NOT Change

The following architectural components are identical to the v2 specification and are preserved without modification:

- All Pydantic V2 schemas: CompanyProfile, ICPSegment, GTMStrategyDocument, MarketingCalendarEntry, SocialPost, AdCopySet, MarketingEmail, VideoScript, VideoScriptScene, ContentPackage, RefinedContent.
- All system prompts and user prompt templates across all four agents.
- Vision 2030 vocabulary context (`_V2030_CONTEXT`) in ResearchAgent.
- Adaptive Querying (build_search_queries with LLM-generated query expansion and static fallback).
- Budget Constraint Engine (`_parse_budget_sar` and 8-channel mathematical allocation in StrategyAgent).
- Context Window Optimiser (`_extract_strategy_anchors` in ContentAgent, ~65% token reduction).
- Self-Reflection Loop (`_critique_draft` and refinement pass in ContentAgent).
- CoT-Aware Audit (Brand Alignment Agent reads ContentAgent reasoning field before making changes).
- Three-layer safety architecture: input validation (15 regex patterns), generation-time guardrails (7 prohibitions), Brand Alignment post-generation audit.
- ChromaDB persistence: 4 collections, PersistentClient, absolute path resolution for Windows compatibility.
- PDF export: fpdf2 with str.translate() Unicode sanitisation, 190mm content width.
- Streamlit session-state substage machine (form / hitl / running).

3. Reliability & Production Readiness

The refactor improves production readiness, not reduces it

Every fallback path from v2 is preserved. The only addition is the ChatOpenAI instrumented layer between app.py and the OpenAI API — a layer that adds observability with no change to output quality or pipeline resilience.

The optional llm parameter on each agent constructor means that if LangChain is unavailable in the runtime environment, the agent automatically falls back to the raw OpenAI SDK. This three-tier fallback (injected llm, local ChatOpenAI, raw SDK) was explicitly designed to prevent the refactor from introducing a hard dependency that could break deployments in constrained environments.

The with_structured_output() calls are wrapped in try/except blocks with fallback to openai_client.beta.chat.completions.parse(). If LangChain's structured output mechanism

encounters an edge case (schema validation failure, model refusal), the pipeline falls back to the raw `parse()` call without surfacing the error to the end user. This dual-path structured output pattern maintains the schema reliability guarantees from v2 while gaining the observability benefits of v3.

The existing `RateLimitError`, `APIConnectionError`, and `APIStatusError` exception handling in `BrandAlignmentAgent` and `ContentAgent` remains fully intact. These exceptions are raised through the `ChatOpenAI` layer identically to the raw SDK layer — `LangChain` does not swallow API exceptions.

4. Conclusion

The v3 refactor completes the `LangSmith` observability integration that was architecturally intended in v2 but was blocked by the raw `OpenAI` SDK call pattern. By introducing a single shared `ChatOpenAI` instance in `app.py` and injecting it as a first-class dependency into all four agents, the system now produces a unified, per-call trace tree in `LangSmith` for every pipeline execution. Engineers can inspect the exact prompt sent to the model, the exact response received, the token consumption split between prompt and completion, the wall-clock latency, and the estimated cost — all without any manual instrumentation overhead.

The architectural cost of this improvement is minimal: four `__init__` signatures gained one optional parameter, four LLM call sites gained one conditional branch, and `app.py` gained three lines in `_get_clients()`. Every other component — schemas, prompts, guardrails, memory, PDF export, UI — is identical to v2. The system is strictly more capable in v3 than in v2, with no regression in output quality, pipeline resilience, or safety compliance.